

Desafío Técnico

¡Hola ! 🙌 *Postulante*

Primero que todo, te agradecemos sinceramente el interés en formar parte del equipo de **(YOL1 o ProntoPaga)**.

Como parte de nuestro proceso de selección, queremos invitarte a desarrollar un desafío práctico. Este ejercicio está diseñado para evaluar tus competencias en:

-  **Backend:** Node.js, TypeScript, diseño de APIs REST y seguridad con JWT.
-  **Frontend:** React y TypeScript.
-  **Buenas prácticas:** Estructura de código, tipado, manejo de errores y arquitectura.



Contexto del Desafío

Imagina que formas parte del equipo de ingeniería de una *Fintech* bancaria. Tu objetivo será construir un MVP seguro para la **Consulta de Riesgo Financiero**, el cual permitirá evaluar el *score* crediticio de personas o empresas según su RUT, aplicando controles de acceso basados en roles.



Objetivo General

1. **Construir una API REST segura** en Node.js + TypeScript que gestione autenticación JWT y autorización basada en roles (`admin / user`).
2. **Desarrollar una SPA** en React + TypeScript que permita el inicio de sesión y la consulta del *score* financiero.



Requisitos Técnicos



Backend (Node.js + TypeScript)

Implementa una API REST con los siguientes endpoints:

1. **POST /login**
 - Simula la autenticación usando credenciales *mock* (no requiere persistencia en base de datos).
 - Retorna un JWT firmado que contenga en su *payload*:
 - `sub`: ID del usuario.
 - `role`: 'admin' o 'user'.
 - `rut`: RUT del usuario (solo si el rol es 'user').

2. GET /score/:rut

- Retorna el *score* financiero de un RUT (un número entre 0 y 100).
- **Regla determinista:** El algoritmo debe devolver siempre el mismo *score* para un RUT específico, pero variar entre RUTs distintos.
- ****Respuesta esperada:****JSON
- {
- "rut": "12.345.678-9",
- "score": 73,
- "fecha": "2025-06-27T14:35:00Z"
- }

Middlewares requeridos:

- **Autenticación:** Validar la firma y expiración del JWT.
- **Autorización:**
 - Rol `user`: Solo puede consultar su propio RUT (debe coincidir con el RUT de su token).
 - Rol `admin`: Puede consultar cualquier RUT.

Frontend (React + TypeScript)

- Interfaz simple, funcional y responsive.
- Formulario de **Login**.
- Vista de **Consulta de Score por RUT**.
- **Manejo de errores:** Notificaciones o mensajes claros cuando un usuario intente consultar un RUT no permitido o ante un fallo de autenticación.

Plazo y Reglas de Entrega

- **Tiempo estimado:** Diseñado para resolverse en un máximo de **3 horas**.
- **Corte de evaluación:** Solo evaluaremos los *commits* realizados dentro del plazo. Te recomendamos hacer *commits* frecuentes y estables a medida que avances.
- **Modalidad de entrega:** Sube tu solución a un repositorio público o privado (con acceso otorgado a nuestro equipo) idealmente en **GitHub** y responde a este correo con el **enlace**.
- **Documentación necesaria:** Incluye un archivo `README.md` con las instrucciones necesarias para ejecutar el proyecto en local (ej. `npm install && npm run dev`).

Uso de Herramientas de Inteligencia Artificial

Está totalmente permitido apoyarte en modelos de IA (ChatGPT, GitHub Copilot, Claude, etc.). Si decides utilizarlos, te pedimos:

 **Transparencia:** Registra brevemente en el `README.md` o en un archivo `ai_interactions.md` qué herramientas usaste y qué partes del código apoyaron a generar.

Queremos entender tu criterio técnico y la forma en que integras asistentes en tu flujo de trabajo.

 **¿Tienes alguna duda sobre el alcance?** Respóndenos a este mismo correo y te responderemos a la brevedad.

¡Mucho éxito, esperamos ver tu propuesta! 🚀